

to manage the resources in the cloud, directly from your machine. The installation of this SDK is straightforward and the link is provided in the *References* section.

Once the SDK is installed, you can specifically use the `gcloud` utility to create and manage VMs in the cloud on which we can have Docker running.

The SDK also ships with multiple client libraries that can be used in popular languages to interact with the cloud platform. In the next section, we will look at how to set up a VM running CoreOS in the Google Cloud, which can then run your Docker loads.

Google Compute Engine

Google Compute Engine is the Infrastructure-as-a-Service (IaaS) offering on GCP. It provides a variety of machine types (CPU, memory), ranging from the basic standard configuration to high-memory instances. It also supports multiple operating system images that can be used to run Docker loads. Specifically, vis-a-vis Docker, it provides custom Google-provided images that come bundled with Docker, rkt and Kubernetes so that you are up and running with your Docker machine as quickly as possible.

One operating system that has been built from the ground up as a container operating system is CoreOS, and Compute Engine supports it while creating the VM. All you need to do is provide a unique VM instance name, the image family (coreos-stable) and image project name (coreos-cloud) while using the `gcloud` command line utility—and within seconds you have a VM with Docker, rkt and Kubernetes running for you.

The steps shown below provide the usage of the `gcloud` SDK:

```
$ gcloud compute instances create container-instance1 \
  --image-family coreos-stable \
  --image-project coreos-cloud
```

Once the above command is run, within a minute you will have a VM ready for you, as indicated by the output below:

```
Created [https://www.googleapis.com/compute/v1/projects/
mindstormcloudemo/zones/us-central1-f/instances/container-
instance1].
```

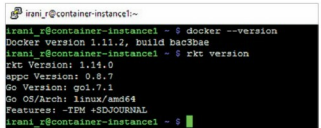
NAME	ZONE	MACHINE_TYPE
container-instance1	us-central1-f	n1-standard-1

```
10.248.0.2      104.155.168.46  RUNNING
```

Now, if we SSH into the container-instance1 IP address, we can see that Docker and rkt are installed and ready for use.

Google Container Engine

Google Container Engine is a container orchestration and cluster manager for running your container workloads. It is based on Kubernetes, the leading orchestration engine for



```
irani_r@container-instance1:~$ docker --version
Docker version 1.11.2, build bac3bae
irani_r@container-instance1:~$ rkt version
rkt Version: 1.14.0
appc Version: 0.8.7
Go Version: go1.7.1
Go OS/Arch: linux/amd64
Features: -TPM +SDJOURNAL
irani_r@container-instance1:~$
```

Figure 1: Google Compute Engine instance with Docker and rkt installed

containers, that was developed by Google itself, before the company open sourced the project on GitHub.

Google Container Engine or GKE, as it is called, is the fastest way to set up a fully managed container cluster in the cloud. Coupled with the fact that it provides support for Kubernetes right out of the box, you get scaling, failover and solutions to all the other challenging distributed software problems that have been addressed by Kubernetes.

The next few commands show how easy it is to get a container cluster running in GKE and to deploy your own service to it.

Assuming that you have the `gcloud` utility installed, creating a container cluster is a breeze. All you need to do is give the following command:

```
$ gcloud container clusters create osfy-cluster
```

It takes a few minutes to create the cluster. By default, a three-node cluster is created with one Kubernetes Master, which is fully managed by GKE itself.

Once the cluster is successfully created, you can view its details, as shown below:

```
Creating cluster osfy-cluster...done.
Created [https://container.googleapis.com/v1/projects/
mindstormcloudemo/zones/us-central1-f/clusters/osfy-
cluster].
kubeconfig entry generated for osfy-cluster.
```

NAME	ZONE	MASTER_VERSION	MASTER_IP
osfy-cluster	us-central1-f	1.4.6	

```
104.154.239.75  n1-standard-1      1.4.6
3
RUNNING
```

Now, let's create a deployment for `nginx` as shown below:

```
$ kubectl run nginx --image=nginx --port=80
deployment "nginx" created
```

We can expose this deployment as a service as follows:

```
$ kubectl expose deployment nginx --type=LoadBalancer"
service "nginx" exposed
```